TRE 4.0 • Computer Science

Deadlock का FORMULA + Semaphore 12 PYQ वाला

Process | Thread | CPU Sched | Deadlock

TRE + UGC NET PYQ • 12 सवाल

@bihar.exam4u

1. Race Condition और Critical Section

Race Condition क्या है

- जब दो या ज़्यादा processes एक ही shared चीज़ (variable/file) एक साथ बदलें, तो नतीजा इस पर निर्भर करता है कि कौन पहले पहुँचा = Race Condition.
- Critical Section (CS) = code का वह हिस्सा जहाँ shared data छुआ जाता है. नियम: एक समय में एक ही process CS में.

CS Solution की 3 शर्तें

- Mutual Exclusion: एक समय एक process.
- Progress: CS खाली हो तो बाहर वाला process दूसरे को न रोके.
- Bounded Waiting: इंतज़ार की सीमा हो.
- कोई assumption नहीं कि CPU कितने हैं या कितनी तेज़ UGC NET 2014

ATM केबिन वाला example

- एक बार में एक आदमी अंदर = Mutual Exclusion
- केबिन खाली है तो अगला तुरंत जाए = Progress
- कोई हमेशा बाहर खड़ा न रहे = Bounded Waiting



PYQ Trap (Dec 2014)

- 'बाहर वाला process दूसरों को block कर सकता है' = यह शर्त नहीं है UGC NET 2014
- Mutual Exclusion: Q CS में तो P और R दोनों बाहर UGC NET 2018
- Strict alternation वाला code: ME मिलता है, पर Progress नहीं UGC NET 2012

2. Synchronization Tools

Semaphore vs Monitor vs Message Passing

Tool	क्या है	Key बात
Semaphore	Non-negative integer	P/wait घटाए, V/signal बढ़ाए
Monitor	High-level package	एक समय एक process अंदर
Message passing	send + receive	System calls, OS के through
Binary sem.	मान 0 या 1	Lock जैसा (mutex)
Counting sem.	कई units	Resource की गिनती

Monitor और Java

- Monitor = higher-level primitive: procedures + variables + data structures एक special package में UGC NET 2014
- Multi-thread में object access control: synchronized methods UGC NET 2014
- Java में synchronized keyword monitor lock लगाता है.

Semaphore के नियम

- P(S): S = S − 1; S < 0 तो process wait में.
- V(S): S = S + 1; waiting process जागता है.
- Negative मान = इंतज़ार करते processes की संख्या.

@bihar.exam4u

3. Semaphore Numerical

Formula

★ अंतिम मान = शुरू − P + V

★ शुरू 7, 20 P और x V, अंत 5

UGC NET 2023

★ 7 − 20 + x = 5 → x = 18

Step by step

- Step 1: P से घटा: 7 − 20 = −13
- Step 2: V से बढ़ा: −13 + x = 5
- Step 3: x = 5 + 13 = 18
- Trap: मान negative हो सकता है

Practice

- शुरू 10, 12 P, x V, अंत 7 → x = 9
- शुरू 5, 8 P, 6 V → अंत = 3
- शुरू 1 (binary), P, P → मान −1, 1 process wait में

याद रखने की ट्रिक

- P = Pichhe (घटाओ), V = Vadhao (बढ़ाओ)
- PV: P-घटा, V-बढ़ा; मान < 0 = लाइन में लोग

@bihar.exam4u

4. Deadlock (गतिरोध)

Deadlock क्या है

- कुछ processes हमेशा के लिए अटक जाएँ, क्योंकि हर कोई वह resource माँग रहा है जो दूसरे के पास है.
- Example: चौराहे पर चारों तरफ से गाड़ियाँ बीच में फँसीं.

Deadlock की 4 शर्तें (चारों साथ हों तभी)

Coffman Conditions

Mutual Exclusion	Hold & Wait	No Preemption	Circular Wait
एक resource एक user	पकड़कर माँगना	छीन नहीं सकते	गोल इंतज़ार
Non-sharable	एक रखो, दूसरा माँगो	खुद छोड़े तभी	P1 से P2 से P3 से P1

Deadlock संभालने के तरीके

तरीका	कैसे
Prevention	4 में से एक शर्त तोड़ो (Havender)
Avoidance	Banker's algorithm, safe state
Detection + Recovery	Process terminate / resource preempt
Ignore	Ostrich algorithm (UNIX/Windows)

@bihar.exam4u

5. Deadlock Formula (बहुत पूछा गया)

Deadlock-free शर्त

★ n processes, हर को अधिकतम m resource, कुल R:

★ R n × (m − 1) + 1

★ यानी सबसे बुरे हाल में भी एक process को पूरा resource मिल जाए.

Trap

- (m−1) भूल जाना सबसे आम गलती.
- n × m R लिखने पर 6 tape drives में n=3 आता है, जबकि सही उत्तर 5 है.

Example 1

- 12 instances, हर process को 4: 12 3n + 1 → n 3 UGC NET 2018
- अधिकतम n = 3

Example 2

- 6 tape drives, हर को 2: 6 n + 1 → n 5 UGC NET 2019
- अधिकतम n = 5

@bihar.exam4u

6. Process/Thread और UNIX के सवाल

Parent-Child

- Parent खत्म तो OS child भी खत्म करे = Cascading Termination
  UGC NET 2022
- Zombie = खत्म process, पर parent ने wait() नहीं किया.
- Orphan = parent पहले खत्म; init adopt करता है.

fork() में क्या share होता है

- fork() के बाद child को stack और heap की अलग copy मिलती है; shared memory segments साझा रहते हैं UGC NET 2016

Match: Algorithm और Area

Algorithm	Area
Stack algorithm	Page replacement
Elevator (SCAN)	Disk scheduling
Priority	CPU scheduling
Havender	Deadlock

2PL vs Timestamp

- 2PL में lock-wait है, इसलिए deadlock संभव UGC NET 2014
  UGC NET 2023
- Timestamp ordering: conflict serializable + deadlock-free
  UGC NET 2014 UGC NET 2023

@bihar.exam4u

7. परीक्षा के लिए Revision

NET में ऐसे आता है

- Statement I/II (2PL vs Timestamp): जहाँ wait, वहाँ deadlock.
- Numerical: semaphore का मान, deadlock-free max n.
- Match the list: Stack/Elevator/Priority/Havend
- Code देखकर: कौन-सी CS property टूटी.

याद रखें

★ Semaphore: P घटाता, V बढ़ाता.
★ Monitor = high-level, एक समय एक process.
★ Deadlock formula में (m−1)

याद रखने की ट्रिक

- CS की 3 शर्तें = MPB (Mutual, Progress, Bounded)
- Coffman = MHNC (Mutual, Hold, No-preempt, Circular)
- R n(m−1)+1 रटो

TRE फोकस

- TRE में परिभाषा वाले और 'कौन-सा नहीं है' वाले सवाल आते हैं.
- Deadlock की 4 शर्तें और CS की 3 शर्तें ज़रूर याद करो.

@bihar.exam4u

@bihar.exam4u

अब Test करो: 12 PYQ

1. Critical section की solution शर्त कौन-सी नहीं है? (UGC NET Dec 2014)

(A) CPU speed की assumption नहीं : यह शर्त है
(B) एक साथ दो CS में नहीं : Mutual Exclusion
(C) बाहर वाला दूसरे को रोके : सही: यह शर्त नहीं
(D) कोई हमेशा wait न करे : Bounded Waiting
Ans: (C) बाहर वाला दूसरे को रोके | Progress: बाहर वाला नहीं रोकता

2. Multi-thread में object access कैसे control होता है? (UGC NET Dec 2014)

(A) Asynchronized methods : ऐसा कुछ नहीं
(B) Synchronized methods : सही: monitor lock
(C) Serialized methods : Serialization अलग चीज़
(D) None of the above : B सही है
Ans: (B) Synchronized methods | Java synchronized = monitor lock

3. Monitor क्या है? (UGC NET Dec 2014)

(A) High-level sync primitive : सही: package
(B) Non-negative integer : यह Semaphore है
(C) send/receive calls : यह Message passing
(D) Block करने वाली calls : IPC calls, Monitor नहीं
Ans: (A) High-level sync primitive | Monitor: procedures+data एक package

हर option का explanation + Answer

Save करो • Share करो • Follow @bihar.exam4u

@bihar.exam4u

Test जारी: सवाल 4-6

4. fork() के बाद parent-child में क्या shared रहता है? (UGC NET Jul 2016)

(A) Heap : Heap की copy बनती है
(B) Stack : Stack की copy बनती है
(C) Shared memory segments : सही: shared रहता है
(D) Heap और Stack दोनों : दोनों की अलग copy
Ans: (C) Shared memory segments | Stack/heap copy, shared mem साझा

5. 12 resources, हर process को 4: अधिकतम n? (UGC NET Dec 2018)

(A) 3 : सही: $12 \geq 3n+1$
(B) 4 : $n=4$ पर deadlock
(C) 5 : $n=5$ बहुत ज़्यादा
(D) 6 : $n=6$ बहुत ज़्यादा
Ans: (A) 3 | $R \geq n(m-1)+1 \Rightarrow 12 \geq 3n+1 \Rightarrow n \leq 3$

6. 6 tape drives, हर को 2: deadlock-free max n? (UGC NET Jun 2019)

(A) 5 : सही: $6 \geq n+1$
(B) 4 : 4 भी चलेगा, max नहीं
(C) 3 : 3 max नहीं
(D) 6 : $n=6$ पर deadlock
Ans: (A) 5 | $6 \geq n(2-1)+1 \Rightarrow n \leq 5$

हर option का explanation + Answer

Save करो • Share करो • Follow @bihar.exam4u

@bihar.exam4u

Test जारी: सवाल 7-9

7. Q CS में है, तो Mutual Exclusion के अनुसार? (UGC NET Dec 2018)

(A) P CS में : ME टूटेगा
(B) R CS में : ME टूटेगा
(C) P और R में से कोई नहीं : सही: कोई नहीं
(D) P और R दोनों CS में : ME टूटेगा
Ans: (C) P और R में से कोई नहीं | एक समय में सिर्फ एक process CS में

8. Semaphore 7, 20 P और x V, अंत S. x = ? (UGC NET Jun 2023)

(A) 15 : 7-20+15=2
(B) 22 : 7-20+22=9
(C) 18 : सही: 7-20+18=5
(D) 13 : 7-20+13=0
Ans: (C) 18 | 7 - 20 + x = 5 => x = 18

9. Parent खत्म तो child भी खत्म: इसे कहते हैं? (UGC NET 2022)

(A) Threading : Thread अलग टॉपिक
(B) Cascading termination : सही नाम
(C) Zombie termination : Zombie अलग है
(D) Process killing : कोई term नहीं
Ans: (B) Cascading termination | Parent खत्म => child खत्म = Cascading

हर option का explanation + Answer

Save करो • Share करो • Follow @bihar.exam4u

@bihar.exam4u

Test जारी: सवाल 10-12

10. Stack/Elevator/Priority/Havender का सही मिलान? (UGC NET 2022)

(A) A-III B-II C-IV D-I : सही मिलान

(B) A-II B-III C-IV D-I : Stack=Page replace

(C) A-III B-II C-I D-IV : Priority=CPU, Havender=DL

(D) A-II B-III C-I D-IV : Stack=Page replace

Ans: (A) A-III B-II C-IV D-I | Stack=Page, Elevator=Disk, Havender=DL

11. 2PL और Timestamp ordering में कौन deadlock-free? (UGC NET 2023)

(A) दोनों : 2PL में deadlock

(B) सिर्फ Timestamp : सही: कोई wait नहीं

(C) सिर्फ 2PL : 2PL में lock-wait

(D) कोई नहीं : Timestamp deadlock-free

Ans: (B) सिर्फ Timestamp | 2PL में wait => deadlock संभव

12. P1,P2 में while(S1==S2) वाला code क्या देता है? (UGC NET Jun 2012)

(A) ME, पर Progress नहीं : सही: alternation

(B) Progress, पर ME नहीं : ME तो मिलता है

(C) न ME न Progress : ME मिलता है

(D) ME और Progress दोनों : Progress नहीं मिलता

Ans: (A) ME, पर Progress नहीं | बारी-बारी: ME हाँ, Progress नहीं

हर option का explanation + Answer

Save करो • Share करो • Follow @bihar.exam4u

@bihar.exam4u

कितना याद रहा?

Comment में अपना score लिखो

Save करो, revision के लिए

BPSC वाले दोस्त को Share करो

Follow @bihar.exam4u - रोज़ नया topic

Detail PDF चाहिए?

Comment में 'PDF' लिखो, DM में भेज दूँगा

कल का topic: OS भाग 4: Memory, Disk, UNIX